Engineering Backlog — Fulfill

Generated from architecture review at commit `b5938ac579a90e5c46acd19c9550835763c66d31` (2026-06-03).
See `docs/architecture-review-2026-06-03.md` for full rationale on each item.

Story points use Fibonacci scale (1, 2, 3, 5, 8, 13). Points reflect implementation effort for a senior engineer including writing tests and verifying in a deployed environment.

P0 — Fix Before Next Production Deploy

- ☐ **Switch release command from `drizzle-kit push` to `drizzle-kit migrate`** `5 pts`
  Generate a baseline migration from the current schema state, commit it, update `fly.toml` release command, verify on a staging deploy. Requires understanding existing schema drift, if any.
  Files: *`fly.toml`*, *`lib/db/`*

- ☐ **Replace `supabase.auth.getUser()` with local JWT verification** `3 pts`
  Install `jose`, extract JWT secret from env, verify token locally in `auth.ts`. Remove the outbound HTTP call from the hot path. Keep `getUser()` for any session-revocation-sensitive paths.
  Files: *`artifacts/api-server/src/middlewares/auth.ts`*

- ☐ **Add unique constraint on `workspaces.owner_id` + atomic upsert** `2 pts`
  Add `.unique()` to the `ownerId` column in the Drizzle schema. Update `ensure-personal` route to use `INSERT ... ON CONFLICT DO NOTHING` pattern. Run migration.
  Files: *`lib/db/src/schema/workspaces.ts`*, *`artifacts/api-server/src/routes/workspaces.ts`*

P1 — Fix Within the Next Sprint

- ☐ **Switch `DATABASE_URL` to PgBouncer endpoint (port 6543)** `1 pt`
  Update the `DATABASE_URL` Fly secret to use the Supabase transaction pooler URL. No code

changes. Verify connections stay under limit under load.
Files: Fly.io secrets only

- [ ] **Move trash-purge scheduler out of the API process** `5 pts`
  Add a dedicated Fly process (`[processes]` in `fly.toml`) or configure Supabase `pg_cron` for the daily purge. Remove `setInterval` from `scheduler.ts` . Ensure only one instance runs per day regardless of machine count.
  Files: `fly.toml` , `artifacts/api-server/src/agents/`

- [ ] **Add composite index `(workspace_id, deleted_at)` on tasks table** `1 pt`
  Add `.index()` to the Drizzle tasks schema. Generate and apply migration.
  Files: `lib/db/src/schema/tasks.ts`

- [ ] **Attach `workspace` to `req` in `requireWorkspaceAccess` ; remove redundant handler checks** `2 pts`
  Extend the `Request` type in `express.d.ts` . Assign `req.workspace` in the middleware after ownership check. Remove the duplicate `if (!req.user)` guards in `columns.ts` and `sprint-snapshots.ts` .
  Files: `src/types/express.d.ts` , `middlewares/requireWorkspaceAccess.ts` , `routes/columns.ts` , `routes/sprint-snapshots.ts`

- [ ] **Write unit tests for `migrate.ts` ID-remapping logic** `5 pts`
  Extract the column → sprint → task ID remapping algorithm into a pure function. Unit test it: correct remapping, parent task IDs patched, predecessor IDs patched, missing-ID edge cases handled, migration fails cleanly without partial writes.
  Files: `artifacts/api-server/src/routes/migrate.ts` *+ new test file*

P2 — Address Before Shared Workspace Ships

- [ ] **Add `workspace_members` table, role enum, and update authorization middleware** `13 pts`
  Schema: `workspace_members(workspace_id, user_id, role, invited_at, accepted_at)` . Backfill current owners as `admin` . Update `requireWorkspaceAccess` to check membership instead of `ownerId` . Audit every route that reads `workspace.ownerId` directly. Update OpenAPI spec and regenerate client. Write tests for each role boundary.
  Files: `lib/db/src/schema/` , `middlewares/requireWorkspaceAccess.ts` , *route files,* `lib/api-spec/openapi.yaml`

- [ ] **Add `helmet` middleware for HTTP security headers** `1 pt`
  Install `helmet` , add `app.use(helmet())` before routes in `app.ts` . Verify CSP doesn't break

frontend asset loading.
Files: `artifacts/api-server/src/app.ts`

- [ ] **Add rate limiting to mutation endpoints** `3 pts`
  Apply `express-rate-limit` to all `POST`, `PATCH`, `PUT`, `DELETE` routes — not just `check-email`. Configure sensible limits per operation class (e.g. 60 creates/min, 120 updates/min). Add `Retry-After` header.
  Files: `artifacts/api-server/src/app.ts` *or per-router middleware*

- [ ] **Cursor-based pagination on collection endpoints** `8 pts`
  Add `?after=<order_value>&limit=<n>` to `GET /tasks`, `GET /sprints`, `GET /columns`. Update OpenAPI spec, regenerate client, update `TaskContext` to handle paginated responses. Add composite index if not already done.
  Files: route files, `lib/api-spec/openapi.yaml`, `artifacts/pm-app/src/app/contexts/TaskContext.tsx`

- [ ] **Split `TaskContext.tsx` into focused contexts** `8 pts`
  Extract sprint logic into `SprintContext`, column logic into `ColumnContext`, leave task-only logic in `TaskContext`. Each context owns its own React Query keys, mutations, and optimistic updates. Update all consumers. Run full test suite after.
  Files: `artifacts/pm-app/src/app/contexts/`

P3 — Nice to Have

- [ ] **Replace optimistic ID construction with `crypto.randomUUID()`** `1 pt`
  Three call sites in `TaskContext.tsx`. Drop the `optimistic-${Date.now()}-${Math.random()}` format.
  Files: `artifacts/pm-app/src/app/contexts/TaskContext.tsx`

- [ ] **Add SIGTERM handler for graceful drain** `2 pts`
  In `index.ts`, listen for `SIGTERM`, stop accepting new connections, drain in-flight requests with a timeout, then exit. Required for zero-downtime Fly.io rolling deploys.
  Files: `artifacts/api-server/src/index.ts`

- [ ] **Audit logging for data mutations** `13 pts`
  Record who changed what, when, on every `POST` / `PATCH` / `DELETE`. Options: a dedicated `audit_log` table (simple, queryable) or structured log events (Pino already in place, no new table needed). Decision needed before implementation: what retention policy, what queries will be run against it.
  Files: new middleware or per-route additions, possibly new schema table

- [ ] **Enforce consistent `req.params` destructuring style** `1 pt`
  Pick one pattern (top-of-handler destructuring) and apply it across all route files. Can be done as part of any other route-touching ticket or as standalone cleanup.
  Files: all `artifacts/api-server/src/routes/*.ts`

Totals by Priority

Individual story estimates (listed per item above) use the Fibonacci scale. The figures below are the arithmetic sum of those estimates per priority group — not individual story point values.

Priority	Items	Sum of Story Points
P0	3	10
P1	4	14
P2	5	33
P3	4	17
Total	**16**	**74**